RETINAI

Diabetic Retinopathy Detection System - Exhibition Guide

Final Year Project for the University of Haripur, Department of Information Technology. This guide is written for exhibition presentation, viva preparation, and quick revision.

---

1. Project Overview

RetinAI is a web-based medical AI system that analyzes retinal fundus images to detect diabetic retinopathy. It helps doctors upload a fundus image, enter patient information, receive a severity prediction, view confidence values, generate an explainable Grad-CAM heatmap, save the scan to a database, and send a report to the patient by email.

The system focuses on five diabetic retinopathy stages: No DR, Mild, Moderate, Severe, and Proliferative. The interface is designed for practical clinic use and exhibition demos rather than only model experimentation.

2. Problem Statement

Diabetic retinopathy is a serious eye complication caused by diabetes. Early detection is important because vision loss can often be reduced if the condition is identified and referred on time. Manual screening is slow, requires specialists, and may be difficult in busy clinics. This project solves that problem by automating the first screening step with AI.

3. Basic Concepts You Should Know

- Diabetic retinopathy is damage to retinal blood vessels caused by diabetes.
- A fundus image is a photograph of the inside of the eye, especially the retina.
- A CNN is a deep learning model used for image classification.
- Transfer learning reuses a pretrained model architecture and adapts it to a new task.
- Ensemble learning combines predictions from multiple models to improve stability.
- Grad-CAM is a visualization method that highlights image regions affecting the prediction.
- Confidence score means how certain the model is about the chosen class.
- Flask is the Python web framework used for the app backend and routing.
- Jinja2 templates render HTML pages dynamically from Flask.
- SQLite stores doctors, scans, and prediction history locally in the project.

4. Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript, Jinja2	Builds the landing page, dashboard, scan form, history page, and report pages.
Backend	Flask	Handles routing, sessions, authentication, and API responses.
AI Models	PyTorch, EfficientNet-B5, ResNet-101	Performs image classification on retinal scans.
Image Processing	OpenCV, PIL, CLAHE	Improves contrast and prepares images for inference.

Layer	Technology	Purpose
Storage	SQLite	Stores users, patient details, scan records, and results.
Reports	ReportLab, HTML email	Creates PDF reports and sends result emails.
UI Assets	Font Awesome, Google Fonts	Provides icons and the visual style used in the pages.

5. Methodology

- Image input: The doctor uploads a retinal fundus image and fills in patient details such as name, age, email, eye side, and medical notes.

- Preprocessing: The image is converted to RGB, enhanced with CLAHE, resized to 256 x 256, normalized, and transformed into a tensor.

- Model inference: EfficientNet-B5 and ResNet-101 are loaded from the models folder when available.

- Ensemble decision: The system averages the probability outputs of both models to produce the final class.

- Interpretability: Grad-CAM is generated to show where the network looked while making the prediction.

- Decision support: The app maps the result to one of five severity levels and attaches clinical recommendations.

- Storage and reporting: The scan is saved in SQLite, can be viewed in history, and can be exported or emailed as a report.

6. System Workflow

Step	What happens
1	Doctor logs in or signs up through the Flask authentication pages.
2	Doctor opens the New Scan page and enters patient details.
3	A fundus image is uploaded from the local system or camera source.
4	The backend preprocesses the image and runs both CNN models.
5	The ensemble result produces a severity label, confidence, and class probabilities.
6	Grad-CAM is generated to make the prediction explainable.
7	The result is stored in SQLite and a detailed report can be viewed or emailed.
8	The dashboard and history pages let the doctor review previous scans and delete records if required.

7. Problems Solved by the Project

- Slow manual screening by giving an instant first-level decision.

- Lack of explainability by showing Grad-CAM heatmaps.

- Single-model instability by combining two deep models in an ensemble.

- Poor follow-up tracking by storing scan history in SQLite.

- Poor communication with patients by generating reports and email output.

- Clinic workflow fragmentation by putting upload, analysis, history, and reporting in one web system.

8. Results and Outcomes

The repository documents the system as an AI-powered five-class diabetic retinopathy detector with a dark-themed web interface, patient record storage, PDF reporting, email delivery, and Grad-CAM explainability. The README states that the trained project targets 90%+ accuracy with the ensemble of EfficientNet-B5 and ResNet-101. If the model files are not available, the app falls back to a demo mode so the full workflow can still be demonstrated during an exhibition.

> Practical outcome: the doctor can demonstrate an end-to-end screening workflow, show the prediction confidence, explain the result visually, and generate a patient-ready report in one session.

> Project result summary: automated screening, explainable AI, patient record management, and report generation were all integrated into one working platform.

9. Conclusion

RetinAI is a complete final-year project that combines computer vision, deep learning, web development, and medical reporting. It addresses the need for quick diabetic retinopathy screening and presents the findings in a form that is easy for doctors to use and easy for examiners to understand. For exhibition purposes, the key message is simple: the system helps detect retinal disease early, explains its prediction, stores the result, and communicates it clearly.

10. Short Viva Talking Points

- Why did you choose diabetic retinopathy? Because early detection is critical and AI can help with screening.
- Why two models? To improve robustness through ensemble averaging.
- Why Grad-CAM? To show that the model is focusing on medically relevant regions.
- Why Flask? It is lightweight and fast for building the web application.
- Why SQLite? It is simple, local, and enough for a final-year project prototype.
- What happens if the model files are missing? The app still runs in demo mode for presentation.

Prepared for exhibition use on 13 May 2026. This PDF was generated from the project workspace.